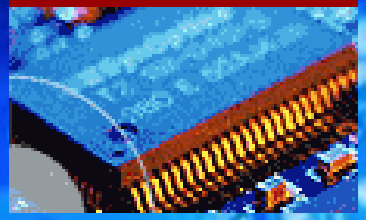
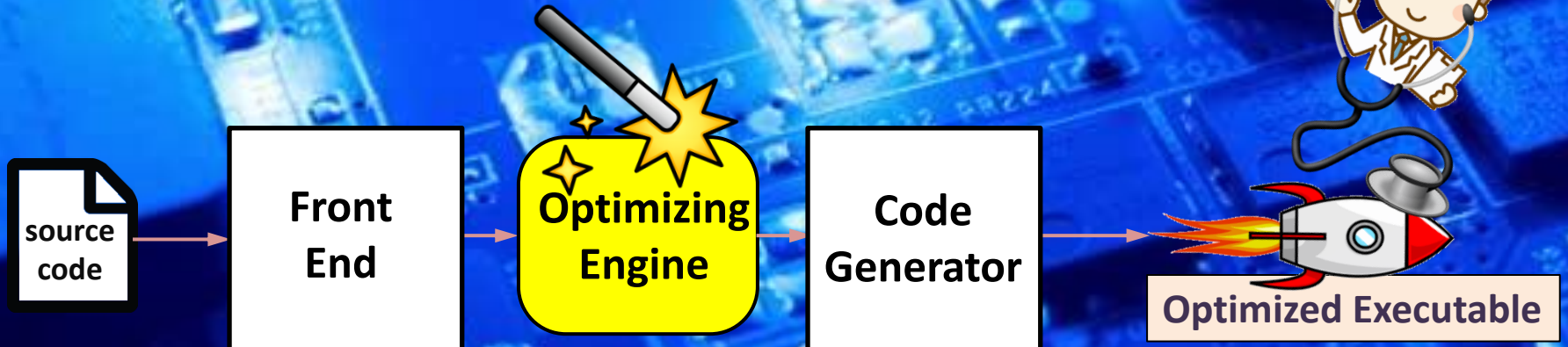
EEE3096S





Compiler Optimization

Lecture L5

Embedded Systems II

Dr Simon Winberg

Electrical Engineering
University of Cape Town

planned as single period lecture

Overview

- Essentials of Optimizing Compilers
- Three Performance Optimization Invariants
- Optimizing Approaches
- Some optimizations to know about
- GCC compiler optimizations and flags

Optional extra reading!

Connection to Text Book* (deepening your knowledge) & lead-in to these slides:
Read over:
Section 2.12 "Translating and Starting a Program". This discusses how C code for a program is translated into executable code that is eventually loaded into memory and jumped to, to execute the program. (You can skip the aspect of Java, it is useful to know about, but it is not covered in this course).
Section 2.13 "A C Sort Example to Put it All Together" gives a nice example explaining the above process.
Section 2.15 from pg 150.e1 elaborates on compiler optimizations, figure e2.15.7 shows the major types of optimizations. Slides for this lecture connect with these techniques, but the objective of these slides is to provide an overview of the essential aspects and awareness of what an optimizing compiler provides and some insight into the approach and design structure of these tools. The availability of a good optimizing compiler is becoming increasingly important to enable quick and efficient use of advanced RISC processors.

*Recommended textbook for this course is "Computer Organization and Design - ARM Edition" by A. Patterson, John L. Hennessy

Compiler Optimization Essentials

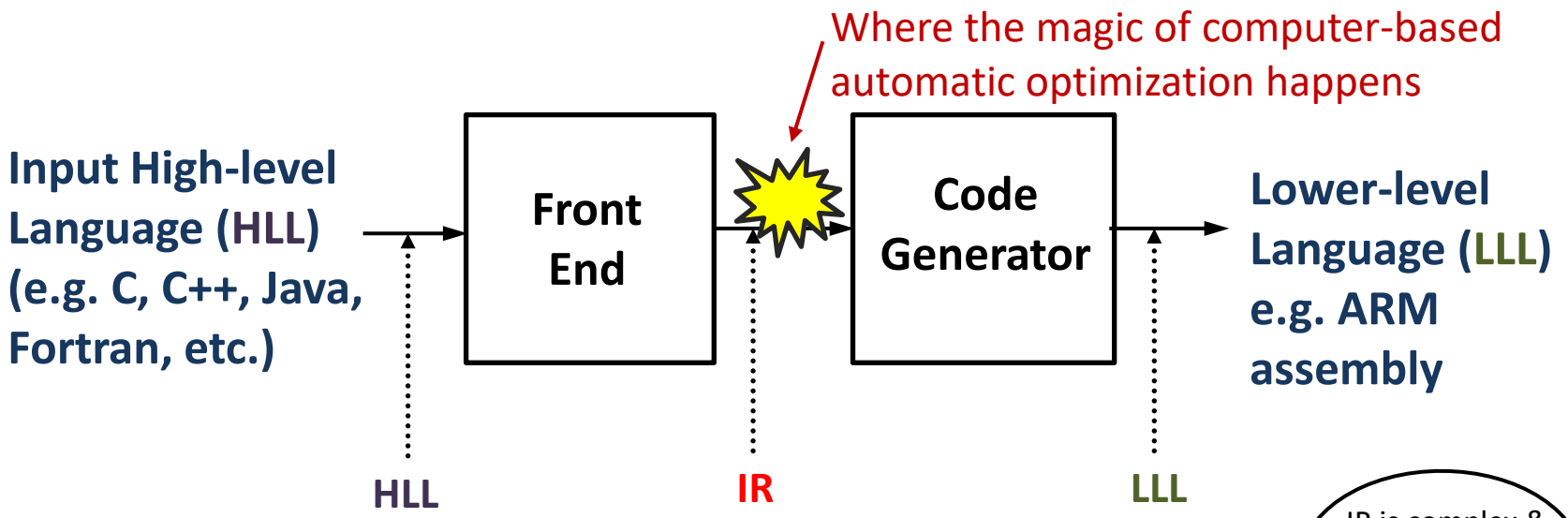
(what an embedded systems engineer should at least know of this)

Brief Background to Compiler Optimization

- Before we simply discuss the GCC optimization flags, it's appropriate for you to know a little (tiny bit) about how an optimizing compiler, such as GCC, works…

Inside an Optimizing Compiler

This is a very simplified view of a compiler structure…

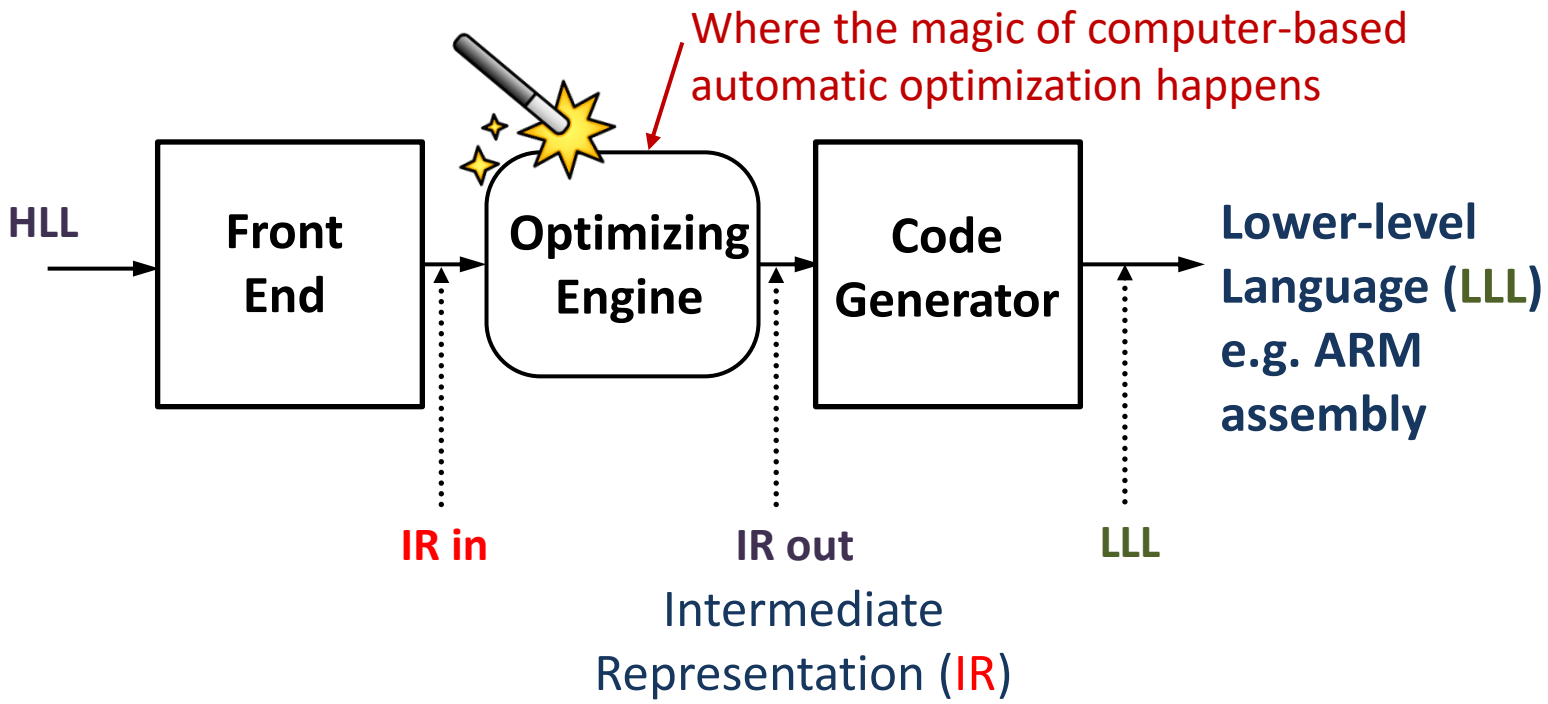


Intermediate Representation (IR) (similar to assembly but may be a bit more high-level and interconnected*)

The 'IR' may be quite a unique language, typically not planned to be human readable, but rather planned around capturing more of the semantic characteristics and associations to other program elements that will make it easier to generate optimize code (I'm personally quite fascinated by these IR languages and they may be designed around the platform and application you are planning to support.)

Inside an Optimizing Compiler

The optimizing engine of a compiler…



The **optimizing engine** is where the incoming IR is reworked, maintaining data dependences and correct operation, etc., to produce a better optimized IR, possibly using the same or different IR language. This is then fed into the code generator that translates the revised IR into lower-level / assembly code (note that this lower-level come is sometimes just C, and this is fed into a more basic, non-optimizing compiler).

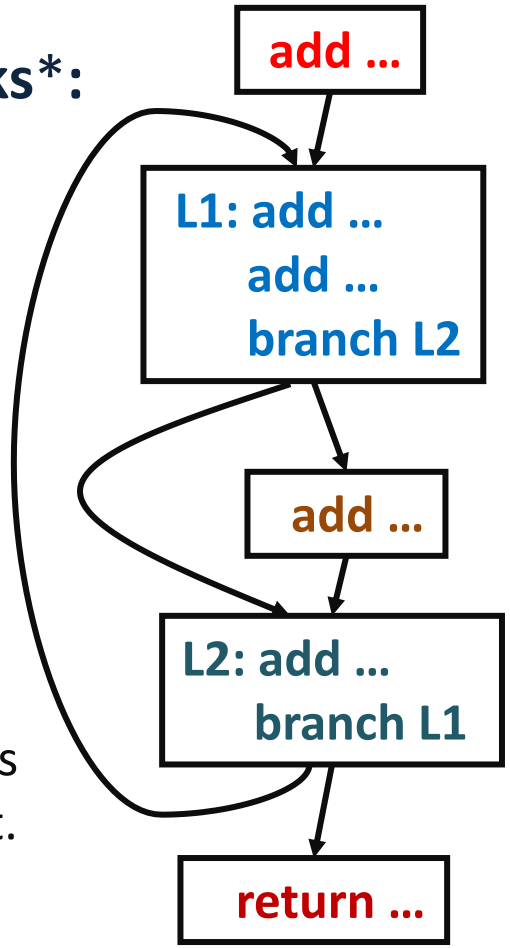
Control Flows (typical used with IR)

The way the compiler 'sees' or works on you program is typically through the use of **control graphs** or control flows. The IR syntax may incorporates the control graph structure (not show in the simple IR illustrated on left).

Example IR:

```
    add …
L1: add …
    add …
    branch L2
    add …
L2: add …
    branch L1
    return …
```

Basic Blocks*:



***Basic Block:** a group of sequential instructions with a single entry point and a single exit point.

Class Activity

Quiz!

https://forms.gle/qpY8DzqJsyZy5ZD78

Use the QR code above to access the quiz on your smartphone.
sample solutions will be presented after the quiz

Three Performance Optimization Invariants

You probably known the term ‘invariants’ *

The three main performance optimization invariants that an optimizing compiler should maintain are the following:

- Though shalt Preserve Correctness
  - (the speed of an incorrect program is likely still worse than a slow one that works correctly)
- Though shalt Improve Performance On Average
  - Optimized may be worse than original if unlucky on some pieces but overall it should be better otherwise it isn’t an optimization
- Though shalt Be Worth It (or be thrown in the NULL bucket abyss)
  - If through the optimizations, the code becomes more fragile, or debugging more difficult – among any other potential drawbacks – then one needs consider if it is worth it either as a user or as the compiler developer.

* i.e. an invariant is a function, quantity, or property which remains unchanged when a specified transformation is applied

Overarching approach: Minimize NoI and CPI

- The thing to keep in mind is:

  execution_time = NOI * CPI

  where: CPI = cycles per instruction*
  and NoI = number of instructions

- Fewer cycles per instruction →
  - Sequence instructions to avoid dependencies and pipelining (e.g. avoid having the next instructing blocking due to it waiting for a result from the previous instruction)
  - Improve cache/memory behaviour (e.g. improving locality, relative addressing, use registers or stack more)
- Fewer instructions
  - Make better use of the available instruction on the target processor (e.g. specialized instructions)

* we'll discuss this further in the computer architecture section in a few lecture's time, we often simplify this down to ACPI, average cucles per instruction, and look at small parts of a program as apposed to the whole thing.

Optimizing Approaches

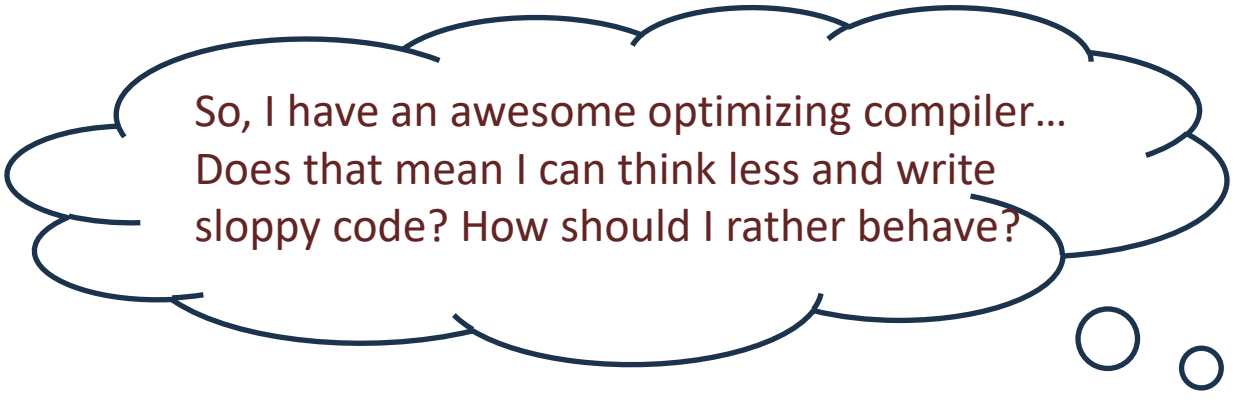
- Efficient mapping of program to the architecture
  - Get rid of minor inefficiencies
  - Ensure efficient code selection and ordering
  - Optimize the register allocation (e.g. sometimes it might be better to have more memory accesses in parts if it means other parts, e.g. a loop has less)
- Allow options for programmer to select best overall algorithm (don't optimize out what the programmer is trying to do*) →

  when in doubt, compiler must be conservative

* This can sometimes be difficult to get right, especially if trying to boost optimization.

Programmer Responsibilities in Using an Optimizing Compiler

Dos & Don't Dos



Responsibility of programmer : Don't Dos

- To don't dos:
  - Smash and grab dirty coding
    - Just because your compiler optimizes things, don't write messy code that is woefully inefficient. Like duplicating lines instead of using a for-loop.
  - Don't write nasty difficult to read code

(this list is a bit short, as it's better to know main problems to avoid and rather focus on what makes for good solutions)

Responsibility of programmer (user)?

- To dos: (I intentionally didn't say 'to do-dos' as it might sound like I'm continuing the previous list)
  - Write readable and maintainable code (very NB!)
  - Select the best algorithm (you know of or can find) … some optimizers might be able to do that behind the scenes but rather put in efficient code.
  - Use procedures if possible (optimizers are better with this) or recursion
  - Eliminate optimization blockers (i.e. things the optimizer has difficulty understanding and likely won't be able to optimization, even though the larger problem may be obvious to a human)
  - Focus on Inner Loops (this is usually the crux of the problem and you can rely on the compiler to use things like loop unrolling to optimize the looping)
  - Put some effort into manually optimizations the code, particularly parts executed repeatedly

Some optimizations to know about

This isn't a compilers course, so you won't be expected to know much about code optimization techniques. However…

As an embedded systems engineer, there are two things you should know about:

1. Small Function In-lining (SFI)
2. Loop Unrolling (LUR)

These two essential ingredients I will briefly explain before we close this section; these are among the many strategies used in GCC's optimization level 2 (invoked with the –O2 flag discussed later)

Small Function In-lining (SFI)

Example scenario:

1. Original code

```
#define DC 3

int addDC (int z) {
  int m = DC;
  return z + m;
}

main(){
  …
  x = addDC(x);
  …
}
```

2. In-lining of simple functions

```
main(){
 …
 {
  int tmp1 = x;
  int m = 3;
  int addDC_r = tmp1 + m;
  x = addDC_r;
 }
 …
}
```

3. Further Optimizing

i.e. dead-code elimination and constant folding. Remove unnecessary assignments.

```
main(){
  …
  x = x + 3;
  …
}
```

assume x above is a global variable int x;

Main Benefits:

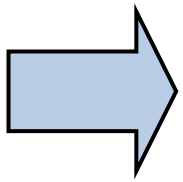
- **Code size:** Can be decreased, and reduce calling overhead, if small procedure body are simply brought into the calling procedure (i.e. 2nd step above)
- **Performance:** eliminates call/return overhead, and can expose the potential further for optimizations (see 3rd step above). But, it can also eat up more instruction memory (trading space for speed)

Optimization: Loop Unrolling (LUR)

Example scenario:

1. Original code

```
j = 0;
while (j < 100) {
    a[j] = b[j+1];
    j += 1;
}
```



2. Loop unrolling

```
j = 0;
while (j < 99) {
    a[j] = b[j+1];
    a[j+1] = b[j+2];
    j += 2;
}
```

And obviously note this also, the 2nd version has ½ the iterations.

Some processors, like ARM, allows the memory address to be added and incremented in one instruction, these instructions are independent so this is better filling up the pipeline with memory reads and writes, that do not depend on each other. This might speed up the loop something like 2x (depending on the pipelining, width of memory bus to cache etc.)

Main Benefits:

- **Reduce looping overhead:** fewer adds to update j, and also fewer loop condition tests
- Allows **more aggressive instruction scheduling**, i.e. more instructions for scheduler to move around

Controlling Optimizations of the GCC Compiler

(mainly this is further reading and to experiment with as part of pracs)

GCC Optimization Flags

There are a great many options and optimization settings that you can set in GCC, a full list can be seen at:

https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

But these are the essential flags that are most commonly used:

Flag	Description
-g:	Include debug information, no optimization
-O0:	Default, no optimization
-O1:	Do optimizations that don't take too long. *Including (look these up if you're interested!):* CP (constant propagation), CF (constant folding), CSE (common sub-expression elimination), DCE (dead code elimination), LICM (loop invariant code motion), ISF (in-lining of small functions)
-O2:	Take longer optimizing, more aggressive scheduling
-O3:	Make space/speed trade-offs: loop unrolling, more in-lining
-Os:	Optimize program size

To use these, simply use e.g.: gcc -O1 main.c -o myprog

End of Lecture